

ABSTRACT

The report summarizes the work performed during an industry internship within the data engineering and analytics domain, specifically the design of scalable and efficient data processing systems. The internship involved the implementation of three large-scale projects in food-tech, retail, and pharmaceutical sectors. All projects covered practical problems of data quality, integration, and analytics.

The initial project was the implementation of an ETL pipeline on Databricks to process semi-structured data with the help of the Medallion Architecture. Cleansing, deduplication, and incremental loading mechanisms ensured data quality, and Delta Lake and Unity Catalog were utilized to ensure reliability, governance, and data lineage.

The second project, the *RetailPulse Analytics Platform*, was aimed at building an AWS-based end-to-end data pipeline. The solution combined data across multiple sources with the help of Apache Spark on EMR, processed datasets were stored in an Amazon S3 data lake, and analytics were enabled with the help of AWS Glue and Athena. Amazon QuickSight was used to create interactive dashboards, along with a serverless real-time anomaly detection system.

The third project dealt with pharmaceutical supply chain issues using real-time streaming with Amazon Kinesis and Spark Structured Streaming. Both continuous monitoring and batch-level traceability were supported by the architecture, which was based on a unified streaming and batch processing design, allowing quick discovery of operational and compliance problems.

Altogether, the internship gave me practical experience in the field of modern data engineering techniques and architectural design patterns, with a focus on scalable platforms, data quality, and real-time analytics to address complex business issues.

CONTENTS

Abstract	vii
List of Figures	x
List of Tables	xi
1 Introduction to Company	1
1.1 About the Company	1
1.2 Location	1
1.3 Operational Structure	1
1.4 Vision and Mission	2
1.5 Products and Software Solutions	2
2 Work on the Project / Case Study	4
Project 1: Zomato ETL Pipeline on Databricks	4
1.1 Introduction	4
1.2 Problem Identification	4
1.3 Objectives	5
1.4 Architecture Overview	6
1.5 Work Carried Out	7
1.6 Solution Provided	9
1.7 Results	10
1.8 Conclusion	11
Project 2: RetailPulse Analytics Platform - End-to-End Retail Data Engineering on AWS	13
2.1 Introduction	13
2.2 Problem Identification	13
2.3 Objectives	14
2.4 Architecture Overview	15
2.5 Work Carried Out	17
2.6 Solution Provided	18

2.7 Results	21
2.8 Conclusion	21
Project 3: Pharmaceutical Supply Chain Streaming Analytics Platform	22
3.1 Introduction	22
3.2 Problem Identification	23
3.3 Objectives	24
3.4 Architecture Overview	25
3.5 Work Carried Out	26
3.6 Solution Provided	30
3.7 Results	31
3.8 Conclusion	33
Bibliography	35
A Plagiarism Report	36
B AI Similarity Report	38
C Project Screenshots	40

LIST OF FIGURES

2.1	Zomato ETL Pipeline Medallion Architecture on Databricks	6
2.2	Driver Notebook - Step-wise Performance and Recording	10
2.3	RetailPulse Analytics Platform End-to-End AWS Architecture	15
2.4	RetailPulse Platform - Spark Processing and BI Result	20
2.5	Pharmaceutical Supply Chain Analytics Platform Kinesis + Spark Architecture	25
2.6	Detailed Architecture of Pharmaceutical Supply Chain Platform	32
2.7	QuickSight Dashboard - Cold Chain Compliance and Regional Sales Analysis	33

LIST OF TABLES

1.1	Infocepts Practice Areas and Functions	2
1.2	Infocepts Proprietary Accelerators and Tools	3
2.1	Medallion Architecture Layer Mapping – Zomato ETL Pipeline	7
2.2	Weekly Work Summary Week 1 - Zomato ETL Project	9
2.3	Zomato ETL Pipeline Test Cases	11
2.4	Technology Stack - Zomato ETL Pipeline	12
2.5	RetailPulse Analytics Platform - AWS Services Used	16
2.6	Weekly Work Summary – RetailPulse Analytics Platform	18
2.7	RetailPulse Analytics Platform - Validation Test Cases	22
2.8	Data Sources and Schemas - Pharmaceutical Supply Chain Platform . . .	26
2.9	Weekly Work Summary - Pharmaceutical Supply Chain Streaming Platform	30
2.10	Pipeline Validation Test Cases - Pharmaceutical Supply Chain Platform .	34

1. INTRODUCTION TO COMPANY

1.1 About the Company

Infocepts is a worldwide data analytics and consulting corporation that focuses on developing scalable data platform, cloud computing, and sophisticated analytics platform to enterprises.

1.2 Location

Infocepts is an organization having a global distributed structure with its corporate base in **Naperville, Illinois, USA**. Its head office of delivery and engineering is situated in Nagpur, Maharashtra, India, which is the hub of data engineering, cloud platform development, and analytics delivery to global customers. Other offices in **Pune, Maharashtra** and **Hyderabad, Telangana** also assist with consulting, pre-sales, and specialty AI/ML services. The business is also located in the USA, New Jersey, to access East Coast customers.

The data engineering, cloud architecture, BI and visualization, quality assurance, and program management are cross-functional teams at Nagpur delivery center where this internship was carried out. The center works across time zones and has real-time collaboration with counterparts in the United States and Europe in enterprise scale data platform projects.

1.3 Operational Structure

Infocepts is organized into competency-based practice areas which collaborate effectively to deliver integrated data solutions. The large functional divisions are:

Each practice area will have a Principal Architect or Practice Head and assisted by senior engineers, data engineers, analysts and associate engineers. It uses agile practices, often in two-week sprints, and daily stand-ups, sprint reviews, and retrospectives, such as Jira, Confluence, and Microsoft Teams.

Interns are integrated right into engineering teams and are exposed to active client deliverables with the help of senior engineers and industry mentors.

Table 1.1: Infocepts Practice Areas and Functions

Sr. No.	Practice Area	Core Function
1	Data Engineering	ETL/ELT pipelines, lakehouse architecture, batch and streaming processing
2	Cloud	AWS, Azure, and GCP infrastructure design and migration
3	Analytics & BI	Dashboards, self-service analytics, KPI frameworks
4	AI & ML	Predictive modeling, NLP, and MLOps pipelines
5	Data Governance	Data quality, lineage, compliance, and cataloging
6	Consulting	Strategy, roadmap definition, and architecture review

1.4 Vision and Mission

The mission of Infocepts is to be a trusted business partner to enterprises in their journey of data-driven transformation to help organizations realize strategic value by using their data assets.

The company works on providing scalable, intelligent and cloud-native data solutions that enable organizations to make decisions that are faster and smarter. It integrates high technical competencies with consulting-based model, making sure that each data platform would provide quantifiable business results and sustainability.

The culture of the organization lays stress on the endless learning process, customer focus, excellence in engineering, and team-based innovation. The professional development of the employees is supported by internal programs including Data Engineering Guild, technical workshops, certification programs (AWS, Databricks, Snowflake), and the mentorship programs.

1.5 Products and Software Solutions

Infocepts is not a conventional product-based company; it provides proprietary accelerators, frameworks and reusable solution templates that supplement its service offerings. The important intellectual property resources are:

Besides these accelerators, Infocepts provides custom solutions on top of industry-leading engines like **Databricks Lakehouse, AWS Glue, Amazon Redshift, Snowflake, Apache Spark Structured Streaming, Amazon Kinesis Data Streams and dbt (data build tool)**. The company also collaborates with such analytics and visualization partners as Tableau, MicroStrategy, and Alteryx.

Table 1.2: Infocepts Proprietary Accelerators and Tools

Sr. No.	Accelerator / Product	Description
1	DataAscent	End-to-end cloud data migration system supporting lift-and-shift and re-architecture patterns across AWS, Azure, and GCP
2	PipelineIQ	Modular ETL/ELT template library built on Databricks and Apache Spark for rapid pipeline development
3	InsightHub	Self-service analytics and BI enablement layer integrating with Tableau, Power BI, and Looker
4	DataQualityX	Data profiling and quality validation platform with rule-based validation and automated alerts
5	StreamEdge	Real-time streaming pipeline accelerator based on Apache Kafka and Amazon Kinesis

Throughout the internship, the intern worked on three projects that dealt with client-facing data engineering, and were in the food-tech industry, retail industry, and pharmaceutical industry. The various combinations of these technologies and accelerators were used in each project. It is discussed in the following chapters in terms of the technical implementations, system architectures, and results.

2. WORK ON THE PROJECT / CASE STUDY

Project 1: Zomato ETL Pipeline on Databricks

1.1 Introduction

Food-tech industry generates vast volumes of semi-structured information daily, such as descriptions of restaurants, customer reviews, type of food, location service, and delivery features. On websites like Zomato, this information is the basis of business intelligence whereby country-level benchmarking, cuisine trend analysis, pricing strategy, and operational decision-making is based on the information.

Nevertheless, the raw data in the form of a JSON are not in a format that can be directly subjected to analysis without a stable, controlled, and scalable transformation pipeline.

During the internship at Infocepts, the aim of this project was to design and develop a production-grade ETL (Extract, Transform, Load) pipeline on Databricks Free Edition and applied the best practices of data engineering. The semi-structured Zomato data is loaded into the pipeline, is processed through various layers, and is stored as Delta Lake tables in Unity Catalog. The end products are refined, subdivided data, which can be used in downstream analytics and reporting.

The answer lies in the Medallion Architecture (Bronze → Silver → Gold), an extremely popular pattern of a lakehouse design that imposes separation of concerns and incremental data quality enhancement on every level. The pipeline is controlled by the centralized driver notebook, which controls execution flow, keeps audit logs, provides concordance control with locking mechanisms, and provides various load strategies, including manual, historical and scheduled execution.

1.2 Problem Identification

Zomato restaurant data is received as nested and multi-level JSON files that have lists of objects of restaurants. Attributes that each object consists of are location, cuisine, pricing, ratings, and delivery information. In its crude form, this data has a few engineering and governance challenges that do not allow it to be consumed analytically directly:

1. Semi-structured Formats: JSON files are highly nested, and restaurant properties are put into multilevel hierarchies (e.g., restaurant.location.city, restaurant.user_rating.aggregate_rati

No access to data in a consistent manner can be made possible without flattening and schema enforcement.

2. **Lack of Data Quality Controls:** Raw data typically include NULL values and missing cuisines, inconsistent string formats, and poorly encoded binary flags. Direct usage in reporting layers results in inaccurate aggregations and unreliable insights.
3. **Duplicate Records:** Reprocessing or ingesting the same records can create duplicates of the records of the same restaurant and file date, which will inflate metrics and give inaccurate business KPIs.
4. **Weak Governance and Lineage:** Tables are hard to find, audit, and manage without centralized metadata management. No formal system exists to trace the history of data origins, transformation, or access.
5. **No Incremental Load Support:** Full refresh strategy of each pipeline execution is compute intensive and inefficient. Systems at the enterprise level must have the ability to load data in an idempotent and incremental way.
6. **Lack of Operational Safety:** Parallel pipelines with insufficient locking can result in overlapping writes and data corruption. Also, debugging and monitoring is challenging because of a lack of audit logs.

These issues form the essence of engineering problem that this project tries to tackle.

1.3 Objectives

The main goals that were set in this project were as follows:

- Use Databricks to run a multi-stage, modular ETL pipeline on the basis of the Medalion Architecture (Bronze → Silver → Gold).
- Use Unity Catalog to govern, manage the schema and discover tables across all pipeline assets in a central point.
- Store the data in Delta Lake, which supports time-travel, scale, and writes that are ACID-compliant.
- Enforce extensive rules of data quality like NULL filtering, deduplication, string normalization and schema enforcement during transformation.
- Calculate business-important columns like rating colour (rating band classification) and cuisines (cuisine category) to add value to analytical data.

- Support incremental, manual and historical load strategies to support operational flexibility, backfills and recovery situations.
- Add audit logging by introducing a special log table (an integer table named zomato_summary_log) and a locking mechanism to guarantee operational observability and concurrency safety.
- Design the pipeline to be run daily, producing country-partitioned and date-partitioned summary tables to be consumed by BI.

1.4 Architecture Overview

The pipeline uses the Databricks Lakehouse platform and adheres to the Medallion Architecture and is structured into three layers of Delta Lake. Unity Catalog is used to control the whole workflow and the driver notebook is used to coordinate it. The high-level architecture is shown in figure 2.1.

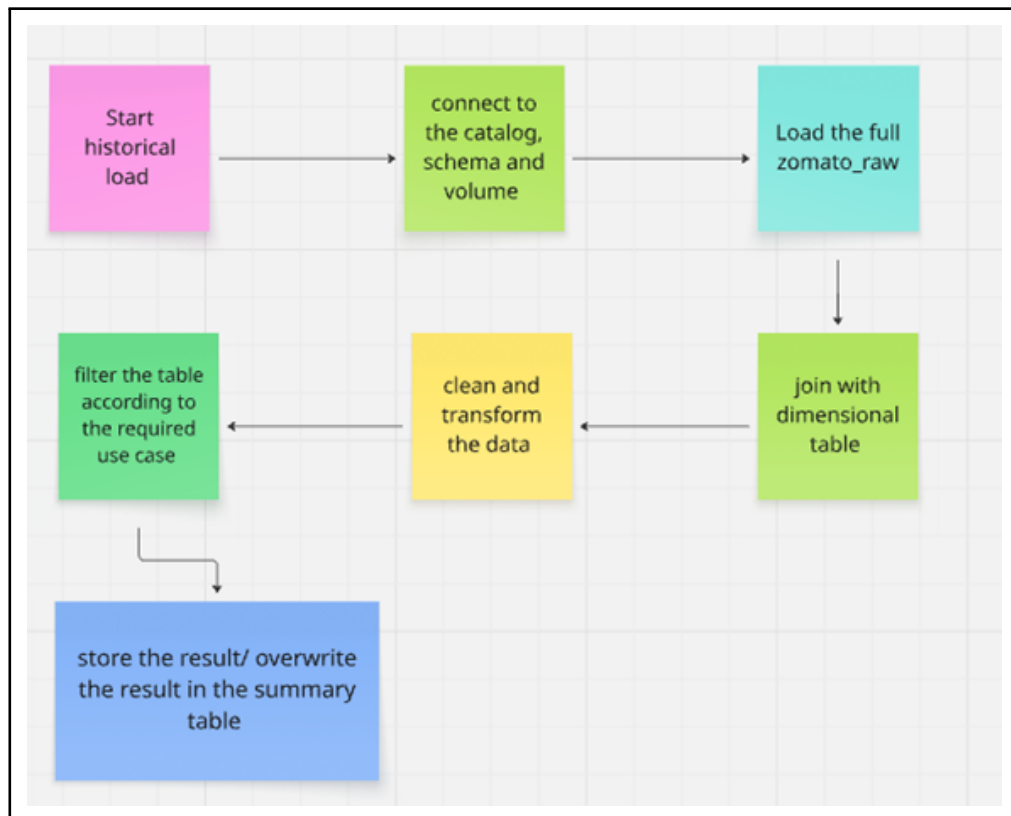


Figure 2.1: Zomato ETL Pipeline Medallion Architecture on Databricks

The structure is divided into four logical layers:

Tier 1 - Ingestion Layer (Databricks Volumes): Raw JSONs are ingested into the folder of a Unity Catalog-managed volume (zomato files) called zomato raw files. The

information is saved in a structured format such as staged JSON files, transformed CSV files, archives and execution logs.

Tier 2 - Bronze Layer (zomato_raw): The ingestion module converts the JSON to CSV which is loaded to the Delta table `zomato.etl.zomato_raw`. This layer contains minimally transformed data which are divided by the file date and serve as the source of the immutable truth of downstream processing.

Tier 3 - Dimension Layer (dim_country): A country dimension table is generated with the help of a reference CSV file. This table adds human readable country names to raw records and allows partitioning of downstream layers by country.

Tier 4 - Gold Layer (zomato_summary): The last analytics-ready table is created by using business rules that include filtering, deduplication, null handling, calculating column derivation, and audit column insertion. The result is saved in a Delta table partitioned on `p_filedate` and `p_country_name`.

Unity Catalog handles all schemas, tables, and access controls under the `zomato` catalog and `etl` schema, guaranteeing the successful governance of all the pipeline.

Table 2.1: Medallion Architecture Layer Mapping – Zomato ETL Pipeline

Tier / Layer	Tier / Table / Asset	Tier / Format	Tier / Purpose
Bronze	<code>zomato.etl.zomato_raw</code>	Delta Lake	Raw ingestion, by file date partitioning
Dimension	<code>zomato.etl.dim_country</code>	Delta Lake	Country lookup (enrichment)
Gold	<code>zomato.etl.zomato_summary</code>	Delta Lake	Analytics-ready, business-rule-applied output
Audit	<code>zomato.etl.zomato_summary_log</code>	Delta Lake	Metadata of ETL execution and step level logging

1.5 Work Carried Out

A summary of the work that was done in this section follows.

The internship input to this project was the entire lifecycle including setting up the environment, module development, orchestration design, and validation. The work was divided into five steps:

Phase 1 - Environment Setup (Environment Setup Notebook): The pipeline environment was configured by setting up the Unity Catalog hierarchy, provisioning the Unity Catalog of the `zomato` catalog and the `etl` schema, and configuring a managed volume (`zomato` files). Raw inputs, staged files, CSV outputs, archives, and logs were

organized into a hierarchical structure of folders. Specific schema definitions, partitioning policies, and storage characteristics were used to create delta tables (`zomato_raw`, `dim_country`, `zomato_summary`, and `zomato_summary_log`). The notebook was idempotent and re-executable in a safe manner.

Phase 2 - Module 1: JSON to CSV Conversion: This module deals with the ingestion stage. The raw landing zone is dynamically identified and copied to a controlled processing path (`source/json`), and the JSON files are read with the multiline JSON reader of PySpark. The embedded restaurant array is blown up and flattened to a tabular format containing 20 business-accepted fields. The filename is used to create a `filedate` column, or the default date is the system date. The DataFrame is processed and then written to one CSV file with the help of `coalesce(1)`. JSON files are processed and stored in form of archived files to prevent reprocessing.

Phase 3 - Module 2: Loading Raw Table and Dimension Table: The resulting CSV files are loaded into the Delta table with schema enforcement into the table named `zomato_raw`. Incremental load plan is executed: incoming data values `filedate` are found, targeted delete statements are used to delete corresponding partitions, new data is inserted. A full overwrite strategy is used to refresh the dimension table (`dim_country`) with a reference CSV, overwriting existing rows and using the standardized column names of the warehouse schema.

Phase 4 - Module 3: Summary Table Transformation and Loading: The dimension table and raw table are joined with country code as LEFT JOIN to ensure that all records are retained. There are seven business rules that are used sequentially: (i) select records whose cuisines are NULL or blank; (ii) filling out string columns with NULLs; (iii) deleting duplicate records using `restaurant_id` and the date of filling records; (iv) inference of rating colour using rating bands; (v) extracting the information about cuisines based on the logic of cuisine categorization; (vi) adding audit columns (`create_datetime`, `user ID`); (vii) renaming columns to partition keys (`p_filedate`, `p_country_name`). The last DataFrame is added to the table `zomato_summary`.

Phase 5 - Orchestration using Driver Notebook: The whole pipeline is controlled by a centralized Driver Notebook. It has runtime widgets to choose the type of load (historical or manual), sends parameters to child notebooks via `dbutils.notebook.run()` and records execution status to the audit table. A locking system is based on files to deny simultaneous executions. Successful runs are stored in a repository and logs are kept with a 7-day archive policy with automated clean-up queries.

Table 2.2: Weekly Work Summary Week 1 - Zomato ETL Project

Week	Activity	Outcome
1	Environment Setup and Architecture Design	Unity Catalog hierarchy, volume, folder structure and Delta table schemas defined and verified
2	Module 1 Development (JSON to CSV)	JSON parsing, extracting fields, derived file-date, generating CSV, and archival logic applied
3	Module 2 Development (Bronze Layer Load)	Incremental partition-level load strategy and dimension table refresh put in place and tested
4	Module 3 Development (Gold Layer Transformation)	Business rules used; summary table generated with partition keys and audit columns
5	Orchestration, Load Strategies, and Validation	Driver Notebook in place; load strategies tested; end-to-end pipeline validated

1.6 Solution Provided

The proposed solution is a governance-first, modular ETL pipeline on Databricks that converts raw Zomato restaurant data in the form of JSON to structured and auditable reverse and analytics-ready Delta Lake tables. The most crucial engineering choices and their business justification are outlined below.

Unity Catalog as the Governance Backbone: The Unity Catalog handles all the pipeline resources, including catalogs, schemas, tables, and volumes (`zomato.etl`). This guarantees centralized access control, metadata management, and data discovery without using uncontrolled storage paths. The active catalog and schema are explicitly spelled out in each notebook to avoid unwanted writes beyond the controlled namespace.

Medallion Architecture to Data Quality Progression: The pipeline is based on a three-layer Medallion Architecture. The Bronze layer (`zomato_raw`) contains raw data, minimally transformed and with the schema, while the Gold layer (`zomato_summary`) has fully processed, analytics-ready data. This division permits safe re-processing and transformations, is modifiable, and may be reused without needing to re-read source data.

Incremental Load Partition-level Overwrite: Rather than complete table overwrites, the pipeline recognizes certain partitions of the incoming data by their respective `filedate` and then removes only those partitions in the Delta table before inserting new records. The method is idempotent, computationally efficient, and can be reliably repeated.

Computed Business Columns: Two calculated columns add value to analysis. The column of aggregate ratings, `m_rating_colour`, is transformed by PySpark into five ratings (Red, Amber, Light Green, Green, Gold). The column `m_cuisines` is defined to fall into categories (e.g., Indian vs. World cuisines) and is matched to keywords. These

enhancements assist filtering and segmentation of BI dashboards intuitively.

Audit Columns and Execution Logging: Audit fields (`create_datetime` and user ID) are attached to each record to record the context of execution (system-generated, manual, or history load). Metadata about pipeline execution is recorded in the table `zomato-summary-log`, which has the job ID, name of step, timestamps, and execution status. This helps in SLA monitoring, failure diagnosis, and compliance auditing.

Concurrency Safety through Lock Mechanism: A file-based locking mechanism is applied to the managed volume. The Driver Notebook generates a lock file early in execution and deletes it in a `finally` block, ensuring release whether successful or not. If an active lock is spotted by another execution, it aborts to avoid concurrent executions and data corruption.

Multiple Load Strategies: The pipeline executes in three modes: (i) *Daily automatic load* to execute, (ii) *Manual load* with Databricks widgets to filter on certain `filedate` or country code, (iii) *Full reprocessing* of available data when required.

This gives it the ability to be used in steady-state as well as recovery/backfill.

Apr 17, 2026, 03:17 PM	Apr 17, 2026, 03:17 PM	...n/environment_setup	23s	✔ Succeeded
Apr 17, 2026, 03:18 PM	Apr 17, 2026, 03:18 PM	...module1_json_to_csv	26s	✔ Succeeded
Apr 17, 2026, 03:18 PM	Apr 17, 2026, 03:19 PM	...n/module2_raw_load	29s	✔ Succeeded
Apr 17, 2026, 03:19 PM	Apr 17, 2026, 03:19 PM	...ission/historical_load	15s	✔ Succeeded
Running Environment Setup Wrote 7 bytes. Starting Zomato ETL Pipeline Running Module 1 Running Module 2 Running Historical Load Archiving processed JSON files Files moved to archive Old logs cleaned Zomato ETL Pipeline Completed Successfully Lock Released				

Figure 2.2: Driver Notebook - Step-wise Performance and Recording

The Driver Notebook runs each of the modules in a predetermined order, logs the state of all steps, and generates a Gold layer output which is country-enriched, rating-classified, audit-enabled, and partitioned to perform optimally on queries (Figure 2.2).

1.7 Results

The pipeline end-to-end testing was completed successfully in all three modes of execution. The following results were systematic and validated:

The raw data was properly ingested into the Bronze table `zomato_raw` where correct partitioning was done by `filedate` and schema was enforced to avoid mismatches in ingesting CSV data. The dimension table was successfully refreshed, and country names were correctly mapped to the appropriate codes in table `dim_country`.

All business rules were properly followed in the Gold table `zomato_summary`. There were no NULL values in the cuisines field, no duplicate combinations of `restaurant_id` and `filedate`, rating color bands and cuisine type were derived properly, and audit columns were fully populated.

Partition pruning was confirmed. Queries using filtering expressions on `p_country_name` or `p_filedate` scanned only the respective Delta partitions, saving computational overhead for country-specific and date-specific analytics.

Simulation of parallel execution tested the concurrency control mechanism. The second attempt of execution was rightly blocked and the failure was registered in the audit log table.

Table 2.3: Zomato ETL Pipeline Test Cases

Test Case Name	Description	Expected Result	Outcome
TC-01	JSON parsing and 20-field extraction	All fields filled in correctly	Pass
TC-02	Filedate derivation from file-name	Assigned correct date per file	Pass
TC-03	NULL cuisine filtering	Zero NULL cuisine records in Gold table	Pass
TC-04	Deduplication on (restaurant id, filedate)	No duplicate rows in summary	Pass
TC-05	Derivation of rating colour	Assignment of rating band correctly	Pass
TC-06	Cuisine classification	Indian / World cuisines appropriately chosen	Pass
TC-07	Incremental partition overwrite	Only target partitions refreshed	Pass
TC-08	Concurrency lock mechanism	Second run aborted and logged	Pass
TC-09	Manual load with filedate filter	Only specified records loaded	Pass
TC-10	Historical load (full overwrite)	Summary table rebuilt with schema intact	Pass

1.8 Conclusion

The Zomato ETL Pipeline project featured an end-to-end, production-grade data engineering solution, based on the Databricks Lakehouse platform. The pipeline successfully met

Table 2.4: Technology Stack - Zomato ETL Pipeline

Sr. No.	Technology / Tool	Role in Pipeline
1	Databricks Free Edition	ETL execution and notebook orchestration platform
2	Unity Catalog	Centralized governance, schema and table management
3	Delta Lake	Storage layer of Bronze, Dimension and Gold tables with ACID compliance
4	Databricks Volumes	Storage of raw, staged, CSV, and archive data
5	PySpark and Spark SQL	Data transformation, filtering, joining and aggregation
6	Python (dbutils, re, uuid)	Utility to orchestrate, file parsing, and job monitoring

major challenges pertaining to data quality, governance, and operational reliability in processing semi-structured Zomato restaurant data by utilizing the Medallion Architecture, Unity Catalog for governance, Delta Lake for ACID-compliant storage, as well as a modular notebook-based orchestration framework.

Engineering-wise, the project offered practical experience with real-world patterns of data platform design, such as partition-level incremental loading, schema enforcement, generation of computed columns, design of the audit trail, and concurrency control. These form the basic skills needed in developing scalable and production-ready data systems.

As a business, the resulting table of data, called `zomato_summary`, allows analytics teams to conduct country-level benchmarking, segmentation based on cuisine, analysis of rating distribution, and evaluation of pricing strategy with a high degree of certainty in the accuracy and completeness of the data.

The multi-load strategy design (scheduled, manual, and historical) provides flexibility and resilience of operations. It not only facilitates regular day-to-day processing, but also specific backfills and whole data recovery conditions without affecting downstream analytics.

All in all, the project had a very solid background in current data engineering practice and architecture design principles, which were also implemented in data pipeline projects in the retail and pharmaceutical sector during the internship.

Project 2: RetailPulse Analytics Platform - End-to-End Retail Data Engineering on AWS

2.1 Introduction

Modern retail and e-commerce companies push huge quantities of transactional information per second, such as customer purchases, inventory updates, partner fulfillment information, and payment events in many systems, formats, and geographies. To convert this raw and unstructured information into valuable, analytics-ready information sets, a scalable, controlled, and automated cloud-native data platform is needed.

In the absence of such infrastructure, challenges that confront organizations include poor reporting of revenues, inventory shortages, slowness in decision making, and general loss of competitiveness.

The RetailPulse Analytics Platform was created as part of the internship at Infocepts as an all-encompassing data engineering platform that emulates how modern retail businesses design and run end-to-end data platforms on Amazon Web Services (AWS). The platform consumes synthetic retail information across a variety of data sources, such as relational databases (Amazon RDS MySQL), semi-structured JSON feeds of external collaborators, and flat-file inventory exports.

The data is handled with a distributed Apache Spark ETL pipeline on Amazon EMR, stored in a layered Amazon S3 Data Lake, and made available as serverless SQL analytics in Amazon Athena on the AWS Glue Data Catalog. Interactive dashboards based on Amazon QuickSight provide business intelligence, while a real-time alerting system is established using AWS Lambda, Amazon SNS, and Amazon SQS.

The platform is designed based on five key principles: scalability, data quality, actionable analytics, multi-source integration, and operational automation. These values are indicative of contemporary production-ready data platform designs in large-scale retail settings.

2.2 Problem Identification

The platform RetailPulse deals with some of the real-life data engineering issues that are often found in retail systems:

1. **Data Quality Degradation:** Raw transactional data frequently has various quality problems. Duplicates can be caused by repeat execution of pipelines or re-extraction of sources and will cause overstated revenue statistics. The absence of important columns like `order_id` and `customer_id` derails joins and aggregations. Orders with dates ahead of the present date (future-dated transactions) misrepresent time-

series analysis and reporting. Poor pricing records, including negative values or unequal discounts, lead to wrong margin calculations.

2. **Multi-Source Schema Heterogeneity:** Data is inputted from heterogeneous sources. Internal RDS tables consume fields such as `order_id`, `customer_id`, and `unit_price`, whereas external JSON feeds consume fields such as `order_ref`, `customer_id`, and `price`. Without schema normalization, it is hard to combine these datasets into a single analytical model.
3. **Absence of Real-Time Monitoring:** Conventional batch analytics systems lack functionality to identify important business events in real time, e.g., stockouts of popular items, order spikes, or sudden declines in order volumes. This leads to sluggish operational responses.
4. **Lack of Cost-Optimized Querying:** Storing data in non-partitioned and non-columnar formats in Amazon S3 results in full scanning of data in response to Athena queries. This adds to the cost and execution time of queries, especially for large datasets.
5. **Lack of Referential Integrity Enforcement:** Orders that contain non-existent customer IDs (orphan records) are not filtered. This results in compromised joins and inaccurate customer-level analytics.

2.3 Objectives

The RetailPulse Analytics Platform was developed with the following engineering and business goals:

- Design a scalable, stratified Amazon S3 Data Lake with clear divisions into Raw, Processed, Quarantine, and Aggregates areas to enable data lineage and repurposing.
- Construct a multi-source ingestion pipeline that ingests structured data (via JDBC) from Amazon RDS and semi-structured partner data (via files hosted in S3) in the form of JSON files, normalizing it to a single schema with PySpark on Amazon EMR.
- Enable Spark Data Quality validation rules, such as detection of null keys, filtering by future dates, window-based deduplication, and orphaned customer detection. Bad records are sent to managed quarantine areas within S3 for governance and investigation.
- Calculate business metrics and enhance validated order data, such as order revenue, discount percentage, profit margin, and weekend transaction flags, and generate an analytics-ready fact table (`fact_orders`).

- Save processed data in partitioned Parquet format to S3 (partitioned by order date) to allow efficient pruning of queries and cost-efficient analytics.
- Autoscale metadata of the processed dataset to schema metadata with AWS Glue Data Catalog and automated crawlers, and query the processed datasets directly using Amazon Athena without the need to manage DDL.
- Provide business intelligence with Amazon QuickSight dashboards, such as daily sales trends, product rankings, category revenue, order status distribution, and geographic revenue analysis with SPICE in-memory accelerations.
- Deploy a serverless alerting pipeline with AWS Lambda, Amazon SNS, Amazon SQS, and Amazon RDS to identify and notify stakeholders about key business anomalies such as stockouts, high rates of cancellations, and zero-order classes.

2.4 Architecture Overview

RetailPulse platform has a cloud-native, multi-phase structure on AWS, encompassing the entire process of data creation to real-time alerting. Figure 2.3 depicts the high-level architecture.

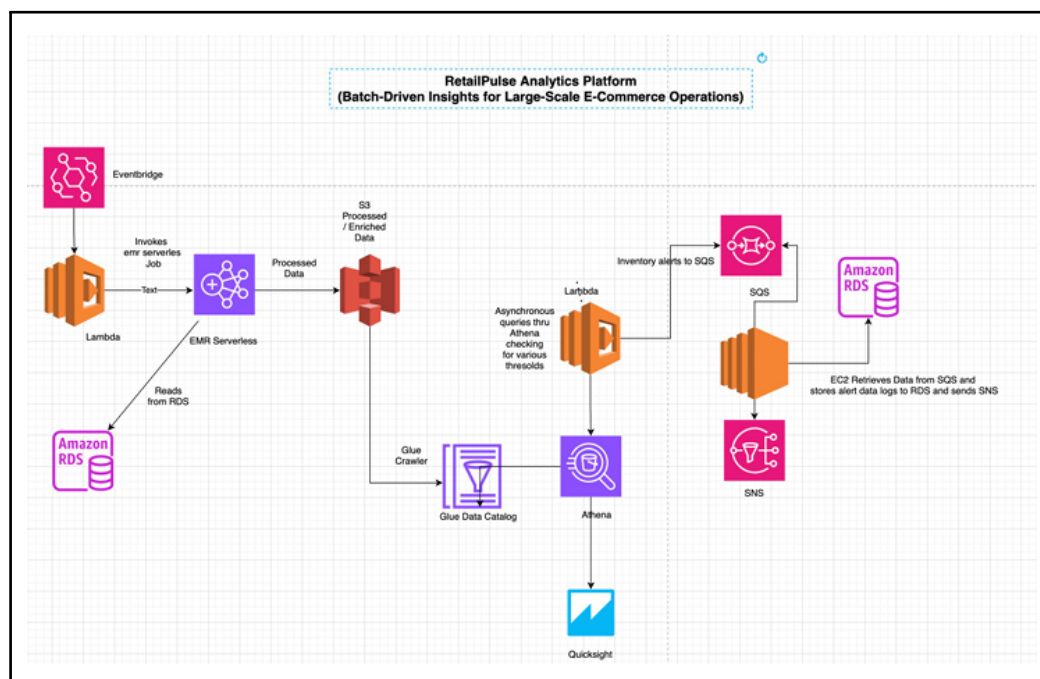


Figure 2.3: RetailPulse Analytics Platform End-to-End AWS Architecture

The architecture is made up of the following stages:

Stage 1 - Data Generation: Python libraries consisting of Pandas, Faker, NumPy and pytz are used to generate synthetic retail datasets. The datasets include customers,

products, distributors, orders (with deliberate data quality problems), inventory snapshots, and external partner orders in JSON format.

Stage 2 - Amazon S3 Data Lake: The S3 bucket (`retailpulse-data-lake`) is designed as four zones: Raw (original files), Processed (clean Parquet datasets), Quarantine (records that have been rejected like null keys, duplicates, and invalid dates), and Aggregates (computed metrics).

Stage 3 - Amazon RDS (MySQL): Structured transactional information is stored in Amazon RDS and loaded by CSV file. It serves as a JDBC source for ingestion in Spark.

Stage 4 - Amazon EMR (Apache Spark): Data in RDS and S3 is read, schema normalized, validated, and business metrics are computed. Enriched data in partitioned Parquet format is written to the Processed zone.

Stage 5 - AWS Glue and Amazon Athena: A Glue crawler finds the schema and records metadata in the Glue Data Catalog automatically. Amazon Athena provides the ability to query SQL without servers and efficiently with partition pruning.

Stage 6 - Amazon QuickSight: This layer offers interactive dashboards based on Athena datasets, with SPICE in-memory acceleration to respond quickly to queries.

Stage 7 - Serverless Alerting: AWS Lambda, Amazon SNS, and Amazon SQS are adopted to execute automated monitoring and alerting processes enabled by scheduled events.

Table 2.5: RetailPulse Analytics Platform - AWS Services Used

Stage	AWS Service / Tool	Role in Platform
1	Python (Pandas, Faker, NumPy, pytz)	Quality defect simulated synthetic data generation
2	Amazon S3	Layered data lake (Raw, Processed, Quarantine, Aggregates)
3	Amazon RDS (MySQL)	Structured relational source; JDBC ingestion point
4	Amazon EMR (Apache Spark)	Distributed ETL processing: validation, transformation, enrichment
5	AWS Glue Data Catalog	Schema discovery and metadata management automation
6	Amazon Athena	Serverless SQL analytics on Parquet data with partition pruning
7	Amazon QuickSight (SPICE)	Business user interactive BI dashboards
8	AWS Lambda, SNS, SQS	Serverless alerting, notification, and message queuing

2.5 Work Carried Out

The internship contribution involved the full lifecycle of the platform across seven stages. Each stage was constructed based on the results of the previous stage.

Stage 1 - Synthetic Data Generation: Six datasets were created using Python scripts. The customer records (1,000 records) were enriched with demographic features like city, state, pincode, customer segment, loyalty level, channel of acquisition, and date of birth using the Faker library (`en_IN` locale) to mimic the Indian retail setup. The product catalogue (200 products) included category, sub-category, brand, MRP, cost price, and re-order levels. The orders dataset (5,000+ records) was created with deliberate data quality compromises: about 5% null records, 3% future-dated records, and 4% duplicates. Other datasets included distributors, inventory snapshots, and partner JSON feeds. Using Pandas, all datasets were exported in CSV/JSON format.

Stage 2 - Amazon S3 Data Lake Setup: An S3 bucket (`retailpulse-data-lake`) was set up in the `ap-south-1` region. A four-zone design was used: `raw/` (landing zone), `processed/` (clean Parquet data), `quarantine/` (invalid records sorted by type), and `aggregates/` (computed metrics). Raw zone was loaded with data via AWS CLI commands (`aws s3 cp`).

Stage 3 - Amazon RDS Setup and Schema Loading: An instance of MySQL 8.x RDS was deployed on a VPC with necessary security groups to connect to EMR (`retailpulse-db`). Five tables (customers, orders, products, distributors, inventory) were established using appropriate DDL schemas, constraints, and primary keys. Files were transferred to an EC2 bastion host and loaded using the `LOAD DATA LOCAL INFILE` command.

Stage 4 - EMR Spark ETL Pipeline: An EMR cluster was used to run a PySpark job. Spark was configured with `spark.sql.shuffle.partitions=200` for optimal parallelism. RDS data was read via JDBC and partner data in S3 was read as JSON. Schema normalization was done by matching column names (e.g., `order_ref` to `order_id`, `cust_id` to `customer_id`). Datasets were combined using `unionByName(allowMi`

Validation rules included: (i) NULL key test with `isNull`, (ii) filtering by future date using current timestamp, (iii) deduplication using window functions, (iv) orphan customer detection via LEFT JOIN.

Invalid records were written in Parquet format to respective quarantine zones. The dataset was supplemented with: `revenue = quantity * unit_price`, `discount_pct = (mrp - unit_price) / mrp * 100`, `is_weekend = dayofweek()`, and profit margin calculations.

Final data was written to `processed/facts/orders/` partitioned by order date.

Stage 5 - AWS Glue and Athena Analytics: A Glue crawler was set up to scan and infer schema, creating the `fact_orders` table in the Glue Data Catalog. Athena queries

included daily revenue using `LAG`, ranking top products with `RANK()`, customer cohort analysis, and inventory risk analysis.

Stage 6 - Amazon QuickSight Dashboards: QuickSight was linked to the Glue Catalog with Athena. Five SPICE dashboards were developed: Daily Sales Summary, Top Products, Category Revenue, Order Status Distribution, and Geographic Revenue Map.

Stage 7 - Lambda Alerting System: An SQS queue (`retailpulse-alert-queue`) and SNS topic (`retailpulse-alert-topic`) were created. Athena queries, business evaluation, and alerting were implemented by a Lambda function (Python, `boto3`). Alerts were posted to SNS and SQS, and stored in an RDS-based table (`alerts_log`). CloudWatch Events scheduled the system (every 5 minutes) to monitor in near real-time.

Table 2.6: Weekly Work Summary – RetailPulse Analytics Platform

Week	Activity	Outcome
1	Data Generation and S3 Setup	Synthetic datasets created; S3 four-zone architecture implemented
2	RDS Schema Design and Loading	MySQL tables created; data loaded; JDBC connectivity validated
3	EMR Spark ETL Development	Validation rules applied; data unified; quarantine zones populated
4	Glue Catalog and Athena Analytics	Schema registered; SQL queries validated; dashboards created
5	Lambda Alerting and Validation	Alert system deployed; end-to-end pipeline validated

2.6 Solution Provided

The RetailPulse Analytics Platform provides an AWS data engineering solution that is production grade. The important engineering decisions include the following that describe the solution and its business value.

Layered S3 Data Lake with Typed Quarantine Zones: Rather than just an accept/reject mechanism, invalid records are sent to semantically structured quarantine sub-folders including: `null_key/`, `future_date/`, and `duplicates/`, which are Parquet-formatted. This makes it possible to perform systematic analysis of data quality problems, trace trends of defects over time, and enhance upstream systems. The raw zone cannot be altered in any way, providing a tamper-proof audit trail and the ability to reprocess it.

Schema Normalization with Column Renaming and `unionByName()`: The names of fields used in partner JSON data are not the same as internal RDS schemas. To deal with schema heterogeneity, the columns are explicitly renamed prior to data merging. Compatibility is achieved by the use of `unionByName(allowMissingColumns=True)`,

which allows the pipeline to be resilient to schema changes when optional fields are not present.

Window Function-Based Deduplication: Window functions are used to find duplicate records by partitioning the data based on `order_id` and sorting by `order_date`. The initial instance is the only one retained. This deterministic method guarantees the same outcome irrespective of data distribution and can be executed in parallel in Spark.

Cost-Optimized Date Partitioning with Parquet Storage: Processed data is stored in Parquet columnar format and partitioned by `order_date`. Amazon Athena uses partition pruning to only scan applicable data, greatly decreasing the cost and time of queries. For example, a query of a 7-day window on a 365-day dataset will save more than 95% of data scanned.

Business Enrichment Measures: Fields of analysis that are calculated in the pipeline:

- `revenue = quantity * unit_price`
- `discount_pct = (mrp - unit_price) / mrp * 100`
- `is_weekend = dayofweek()`
- `profit_margin = revenue - cost_price`

Such enrichments make it possible to consume the analysis directly without further joins.

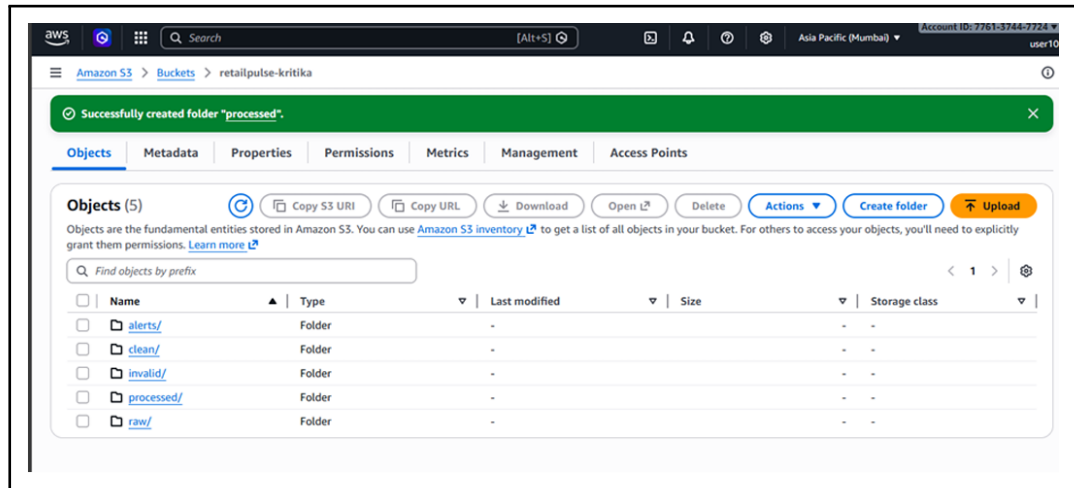
Advanced Athena Analytics: SQL window functions used in Athena queries include: `LAG()` to get day-over-day revenue growth and `RANK()` to rank products. Inventory risk classification (e.g., AT RISK vs STABLE) can be done using conditional logic with the `CASE` expression, which is directly related to alerting workflows.

Fault-Tolerant Serverless Alerting: The Lambda–SNS–SQS architecture guarantees the delivery of alerts. AWS Lambda runs monitoring queries, SNS delivers live notifications, and SQS guarantees message durability and re-run capability. Alerts are stored in the table `alerts_log` in RDS for auditing and trend analysis.

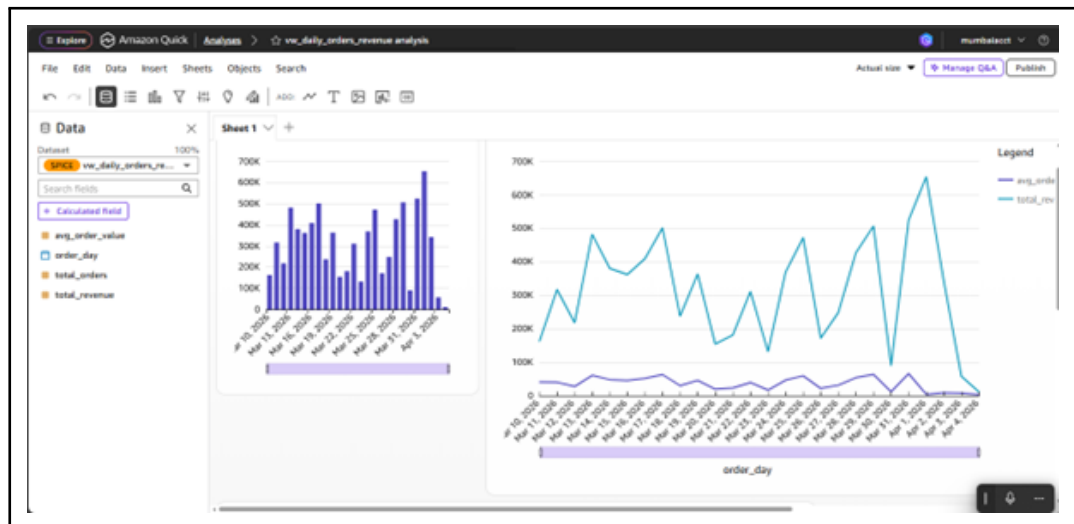
There are four important types of alerts:

- Stockouts of best-selling products
- Large cancellation rates (e.g., >25%)
- Zero-order category detection
- Decrease in daily revenue

As Figures 2.4 depict, the Spark ETL layer cleanses the data in sequential validation phases and then enriches it, whereas the QuickSight layer presents clean and analytics-ready data in interactive dashboards.



(a) S3 Data Lake – Validation and Quarantine Routing



(b) QuickSight Dashboards – Sales Summary and Category Revenue

Figure 2.4: RetailPulse Platform - Spark Processing and BI Result

2.7 Results

The end-to-end testing was performed on all seven stages. The following results were validated systematically at query-level:

Information was properly divided into all four areas of the S3 Data Lake. The quarantine layer captured invalid records such as those with NULL keys, future-dated records, and duplicates. The records were added to their respective subfolders in Parquet format and checked with Athena queries.

The processed `fact_orders` table had zero null critical keys, zero future-dated records, and no duplicate `order_id` records. All enrichment columns were properly calculated and saved (revenue, discount percentage, `is_weekend`, profit margin).

Partition pruning was successful. An Athena query that ran for 7 days on a 90-day dataset only read the partitions it needed, drastically decreasing the amount of data scanned and the query cost. Elaborate Athena queries yielded precise outcomes, such as best product listings, revenue patterns, customer cohorting, and inventory risk detection.

SPICE successfully loaded QuickSight dashboards, and all five dashboards displayed correct and interactive visualizations. The Lambda alerting system identified injected anomalies, issued SNS notifications, sent messages to SQS, and recorded alerts in the RDS table `alerts_log`.

2.8 Conclusion

The RetailPulse Analytics Platform is able to implement a full, production-grade, cloud-native data engineering solution on AWS. It addresses major issues in data quality, multi-source integration, scalable storage, cost-effective analytics, business intelligence delivery, and automated monitoring in a retail setup.

As an engineer, the project provided pragmatic experience with AWS services, including EMR, RDS, S3, Glue, Athena, QuickSight, Lambda, SNS, and SQS. It also strengthened fundamental data engineering principles such as schema normalization, deduplication via window functions, data quality management via quarantine, and partitioned columnar storage optimization.

Business-wise, the platform transforms reporting from reactive to data-driven and proactive decisions. QuickSight dashboards provide actionable insights into sales trends, product performance, and customer behavior. The alerting system is real-time, ensuring that important operational events such as stockouts, high cancellation rates, and demand anomalies are reported within minutes.

The architecture is very flexible and scalable. With suitable adjustments in EMR cluster settings, Spark partitioning, and Glue crawler schedules, the platform can support spikes in data volume without requiring any structural adjustments.

Table 2.7: RetailPulse Analytics Platform - Validation Test Cases

Test Case	Description	Expected Outcomes	Status
TC-01	NULL order id / customer id detection	Records sent to quarantine/null_key/	Pass
TC-02	Future-dated order filtering	Records sent to quarantine/future_date/	Pass
TC-03	Duplicate order deduplication	Only first occurrence retained	Pass
TC-04	Orphan customer detection	Missing customers flagged, not dropped	Pass
TC-05	Schema normalization (JSON + RDS)	Unified schema applied correctly	Pass
TC-06	Revenue and discount calculations	All calculated values correct	Pass
TC-07	Partition pruning in Athena	Only required partitions scanned	Pass
TC-08	Glue crawler schema inference	fact_orders registered correctly	Pass
TC-09	QuickSight SPICE ingestion	Dashboards displayed successfully	Pass
TC-10	Lambda alert (high cancellation)	SNS, SQS, and RDS logging successful	Pass
TC-11	Lambda alert (top-seller stockout)	Alert triggered correctly	Pass
TC-12	Lambda alert (zero-order category)	Alert generated correctly	Pass

In general, this project illustrates the construction and execution of a modern, scalable, and intelligent retail data platform in compliance with business data engineering practices.

Project 3: Pharmaceutical Supply Chain Streaming Analytics Platform

3.1 Introduction

Pharmaceutical supply chains are highly regulated, quality- and operation-intensive environments that traditional data platforms are ill equipped to manage. Medicines with temperature-sensitive properties like vaccines, insulin, biologics, and controlled substances must be under constant cold chain supervision during manufacturing, storage, distribution, and dispensing.

Each manufactured batch must have complete traceability, linking manufacturing records, quality control test results, distributor inventory movements, and pharmacy sales transactions. Organizations need to measure the impact of recalls—whether on orders, in transit, or revenue value—in hours, not days, in the event of a post-release quality failure.

Traditional pharmaceutical data systems do not address these needs because of their siloed and batch-based character. Temperature variances are usually identified with delays, inventory assessment is based on manual operations, visibility differs among distributors, and quality control information is not combined with real-time operations.

This project, as part of the internship at Infocepts, offers an end-to-end, event-based pharmaceutical supply chain analytics platform. To overcome these challenges, the platform combines real-time streaming, batch processing, and built-in data modeling.

Real-time data ingestion is done through Amazon Kinesis Data Streams, which include temperature sensor results, pharmacy sales, and distributor inventory updates. Spark Structured Streaming is used on Amazon EMR to process these streams in real time and store them in an S3 data lake, managed with Hive.

Apache Sqoop is used to ingest batch data (product master, batch master, inventory snapshots, and QC test results) into Hive tables, with multiple layers (raw, clean, valid, and invalid). The shared `batch_id` serves as the central linking key across all datasets to create a unified data model.

The final analysis layer is driven by Amazon Athena and visualized with QuickSight dashboards. This helps stakeholders track cold chain compliance, inventory risk, batch traceability, and near real-time evaluation of recall impacts.

3.2 Problem Identification

The pharmaceutical supply chain analytics platform was developed to respond to four key business and engineering issues:

1. **Lack of Real-Time Cold Chain Monitoring:** Pharmaceutical products sensitive to temperature must be stored and transported under rigid conditions (e.g., 2–8°C). IoT sensors measure temperature values frequently (e.g., every 30 seconds). Without real-time streaming systems, manual audits detect excursions only after products have moved through the supply chain, leading to compliance risks and potential patient harm.
2. **No Batch-Level End-to-End Traceability:** All units dispensed should be trackable to the manufacturing lot, including QC, temperature history, and supply chain path. However, data is spread across ERP (manufacturing), LIMS (quality), WMS (distribution), and POS (pharmacy). End-to-end traceability is difficult without a standard linking mechanism. The natural key to combine these datasets is the `batch_id`.
3. **Fragmented Inventory Visibility:** Inventory information in distributors and warehouses is not connected, preventing real-time monitoring of inventory levels. With-

out constant tracking of stock coverage and demand trends, replenishment decisions are reactive, leading to stockouts or near-expiry inventory.

4. **Delayed Quality Failure Impact Assessment:** In the case of a failed batch in quality testing (e.g., abnormal potency, sterility problems, endotoxin contamination), identifying affected orders, inventory locations, and revenue at risk is manual and time-intensive. Such delays (typically 24–72 hours) increase the risk of defective products reaching patients.

3.3 Objectives

The following are the engineering and compliance goals of the Pharmaceutical Supply Chain Analytics Platform:

- Use an Amazon Kinesis Data Streams ingestion pipeline and Spark Structured Streaming on Amazon EMR to continuously ingest temperature sensor events, pharmacy sales transactions, and distributor inventory events with a latency of less than one minute.
- Fully validate the data at the stream level such as temperature unit conversion (Fahrenheit to Celsius), range validation (-50°C to 100°C), positive sales quantity and price validation, and inventory change type validation. Valid records are kept and invalid events are eliminated.
- Construct a batch ETL pipeline with Apache Sqoop to import manufacturing batch master, product master, QC results, and inventory snapshots from Amazon RDS MySQL into Hive.
- Develop a common data spine using `batch_id` in all datasets to enable end-to-end traceability between temperature, sales, inventory, and QC data.
- Provide near real-time impact analysis of QC failures by associating QC failures with live orders and inventory.
- Conduct six Athena queries on cold chain compliance, risk of stockout, effect of recall, distributor violations, abnormal sales trends, and QC compliance.
- Construct SPICE-based QuickSight dashboards to monitor the cold chain, inventory risk, and operational insights.

3.4 Architecture Overview

The platform has two parallel processing lanes: a real-time processing lane (streaming lane) and a batch processing lane, both of which meet at the Hive metastore and are accessed via Amazon Athena and visualized through QuickSight.

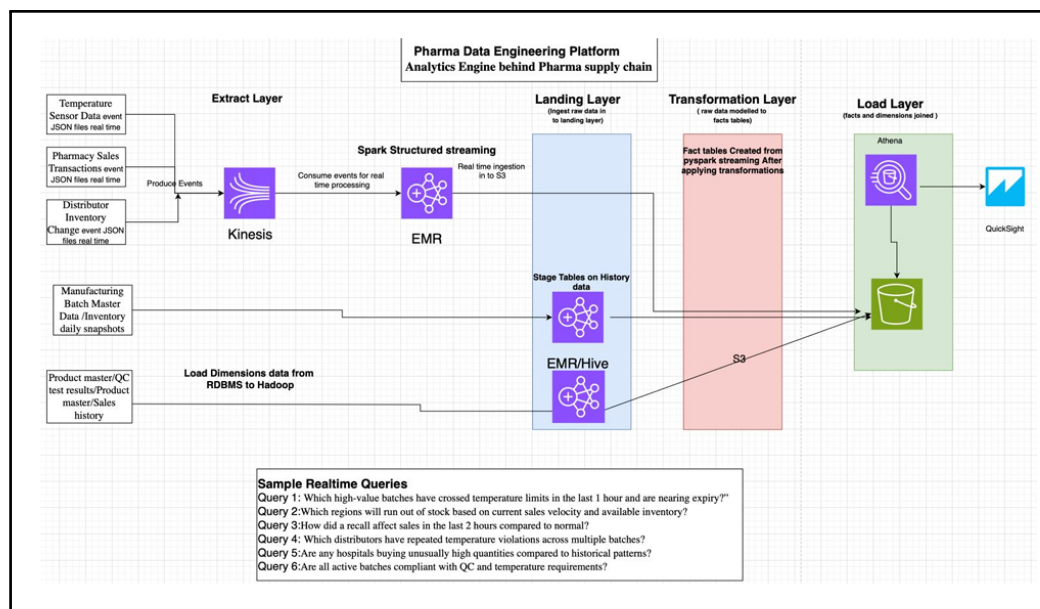


Figure 2.5: Pharmaceutical Supply Chain Analytics Platform Kinesis + Spark Architecture

Streaming Lane:

A Python script (`producer.py`) running on an EC2 instance generates three event streams: temperature sensor readings, pharmacy sales transactions, and distributor inventory updates.

The producer reads CSV files, assigns a `source_type`, shuffles records, and publishes them to a Kinesis Data Stream (`pharma_kinesis_kritika`) at a 30-second rate using `boto3`.

A PySpark Structured Streaming application (`consumer.py`) on EMR:

- Reads data from Kinesis with the `spark-sql-kinesis` connector.
- Parses JSON using `from_json()`.
- Divides data into three streams based on `source_type`.
- Applies validation and transformations.
- Writes to Hive-backed S3 Parquet tables.

Micro-batch period: 120 seconds with checkpointing.

Batch Lane:

Four tables are extracted from Amazon RDS MySQL using Sqoop import: batch master, product master, QC results, and inventory snapshots.

Each dataset is processed in Hive by:

- `_raw` – raw ingestion
- `_clean` – clean up and type casting
- `_valid` – business-rule validated data
- `_invalid` – invalid data

Verified data is written to S3 in Parquet format for querying with Athena.

Analytical Layer:

Amazon Athena queries the S3 Parquet datasets using Glue Catalog (`pharma_db_kritika`). Six analytical queries combine both batch and streaming data using `batch_id`.

QuickSight takes these datasets through SPICE to generate dashboards.

Table 2.8: Data Sources and Schemas - Pharmaceutical Supply Chain Platform

Source	Dataset	Transport	Key Fields
Streaming	Temperature Sensors	Kinesis (30 sec interval)	sensor_id, batch_id, temperature_c, alert_flag
Streaming	Pharmacy Sales Transactions	Kinesis (real-time)	transaction_id, batch_id, revenue_inr, fulfillment_status
Streaming	Distributor Inventory Changes	Kinesis (real-time)	event_id, batch_id, change_type, current_stock
Batch	Manufacturing Batch Master	S3 (daily)	batch_id, min_temp_c, max_temp_c, batch_status
Batch	Product Master	RDS (daily)	product_sku, cold_chain_required, controlled_flag
Batch	QC Test Results	LIMS (daily incremental)	qc_id, batch_id, overall_result, potency_pct
Batch	Inventory Snapshot	WMS (daily)	snapshot_date, distributor_id, closing_stock, reorder_point

3.5 Work Carried Out

The internship contribution involved the full implementation of both streaming and batch pipeline lanes, along with Athena analytics and QuickSight visualization.

Phase 1 - Kinesis Stream Provisioning and EC2 Producer Setup: A Kinesis Data Stream named `pharma_kinesis_kritika` was provisioned in on-demand mode in the ap-south-1 region to accept variable throughput without manual sharding.

An EC2 instance (t3.micro, Amazon Linux 2023) was started and set up with Python 3, boto3, and pandas using `pip install`. AWS CLI credentials were configured for secure access.

Three SCP-uploaded datasets were used: `temperature_events.csv`, `sales_transactions.csv`, and `inventory_events.csv`.

The `producer.py` script:

- Reads all datasets.
- Adds a `source_type` field.
- Combines and rearranges records.
- Pushes records to Kinesis with `kinesis.put_record()`.
- Randomly partitions records and sleeps using `time.sleep(30)`.

The validation of streaming was carried out by terminal logs that showed successful records transmission.

Phase 2 - EMR Spark Structured Streaming Consumer:

Kinesis connector JAR (`spark-sql-kinesis-2.12-1.2.0_spark-3.0.jar`) was loaded on the EMR cluster.

The `consumer.py` application:

- Creates a `SparkSession` with Hive support.
- Reads Kinesis stream with: `spark.readStream.format("aws-kinesis")`
- Converts binary to string: `CAST(data AS STRING)`
- Parses JSON with `from_json()`
- Divides streams using `source_type`

Stream-specific transformations:

- **Temperature Stream:** Transforms Fahrenheit to Celsius; acceptable range (-50°C to 100°C).
- **Sales Stream:** Casts quantity and price, removes invalid values.
- **Inventory Stream:** Validates change type based on whitelist.

The partition column was obtained as `partition_date` using `to_date()`.

Data was written to Hive-managed Parquet tables:

- `pharma_streaming_db.temperature_streaming`
- `pharma_streaming_db.sales_streaming`
- `pharma_streaming_db.inventory_streaming`

Streaming jobs used:

- Micro-batch interval: 120 seconds
- S3-based checkpointing

Execution:

```
spark-submit --jars spark-sql-kinesis_2.12-1.2.0_spark-3.0.jar consu
```

Spark SQL queries were used to validate.

Phase 3 - Batch Pipeline: Sqoop Extraction and Hive Processing:

MySQL Connector/J was added to Sqoop libraries.

Sqoop imports:

- Retrieved product master and QC results
- Used JDBC connection to Amazon RDS
- Stored data in HDFS as text files

Hive processing involved 4 layers:

- `_raw`
- `_clean`
- `_valid`
- `_invalid`

Data cleaning techniques:

- `CAST()`, `TRIM()`, `NULLIF()`
- Uppercase standardization
- Validation using CASE conditions

Example validation:

- `overall_result IN ('PASS', 'FAIL')`

- potency_pct BETWEEN 80 AND 120

Results:

- Total QC records: 200
- Valid: 183
- Invalid: 17

Sound data was then saved to S3 in Parquet format:

s3://pharma-data-lake-kritika/hive_exports/pharma_db_kritika/

Phase 4 - Athena Analytics and QuickSight Dashboards:

Athena was set up with Glue Data Catalog (pharma_db_kritika).

Six analytical queries were implemented:

- Cold chain compliance violations
- Stockout risk identification
- Recall impact analysis
- Distributor compliance tracking
- Abnormal sales pattern detection
- Batch compliance assessment

QuickSight integration:

- Linked through Athena data source
- SPICE datasets created
- Dashboards built:
 - Cold Chain Compliance Analysis
 - Regional Sales Performance

Table 2.9: Weekly Work Summary - Pharmaceutical Supply Chain Streaming Platform

Week	Activity	Outcome
1	Kinesis Provisioning & EC2 Producer Development	Stream generated; producer publishes events every 30s; verified with logs
2	EMR Spark Streaming Consumer	Added three streams to Hive tables; checkpointing verified
3	Sqoop Extraction & Hive Processing	Batched ingested data; implemented four-layer Hive model
4	S3 Export, Athena Analytics & QuickSight	Parquet export completed; Athena queries executed; dashboards published

3.6 Solution Provided

The platform offers four closely interconnected technological functions which together address significant challenges in the pharmaceutical supply chain.

Concept: Kinesis and Spark Structured Streaming for Real-Time Cold Chain Tracking: IoT temperature data is sent every 30 seconds to Amazon Kinesis and processed in 120-second micro-batches with Spark Structured Streaming.

Temperature normalization:

$$T_C = (T_F - 32) \times \frac{5}{9}$$

A validation filter ensures values are within the physical range (-50°C to 100°C). Batch master thresholds (min_temp_c, max_temp_c) trigger an alert flag when temperature is out of range. This enables near real-time (under 3 minutes) cold chain breach detection.

Unified Batch-Centric Data Spine with batch_id: Each dataset is joined on the shared key `batch_id`. The analytical layer joins:

- Temperature valid with batch valid for compliance tracking
- Sales valid with batch valid to link transactions
- Inventory valid with batch data to analyze stock

This enables full end-to-end manufacturing-to-dispensing traceability.

Batch Processing: Four-Tier Hive Architecture: Each dataset is processed through four layers:

- `_raw` - raw HDFS data (immutable)
- `_clean` - type casting, trimming, null handling
- `_valid` - business-rule filtered data

- `_invalid` - rejected records

Example validation rules:

- `potency_pct BETWEEN 80 AND 120`
- `overall_result IN ('PASS', 'FAIL')`

Of 200 QC records:

- Valid: 183 (91.5%)
- Invalid: 17

Automated Recall Impact Analysis: Athena queries join sales valid with batch valid where:

```
batch_status = 'RECALLED'
```

Revenue in recall cases is matched against historical baselines, enabling rapid impact evaluation within seconds.

3.7 Results

The platform has been proven to work with both streaming and batch pipelines.

Streaming Results:

- Kinesis producer created interleaved temperature, sales, and inventory events.
- Spark consumer properly split streams and applied validations.
- Data written to Hive tables:
 - `temperature_streaming`
 - `sales_streaming`
 - `inventory_streaming`
- Partitioned Parquet files stored in S3: `streaming_kinesis/temperature/, sales/, inventory/`

Batch Results:

- Sqoop successfully extracted data from RDS to HDFS.
- Hive pipeline produced:
 - 200 overall QC records

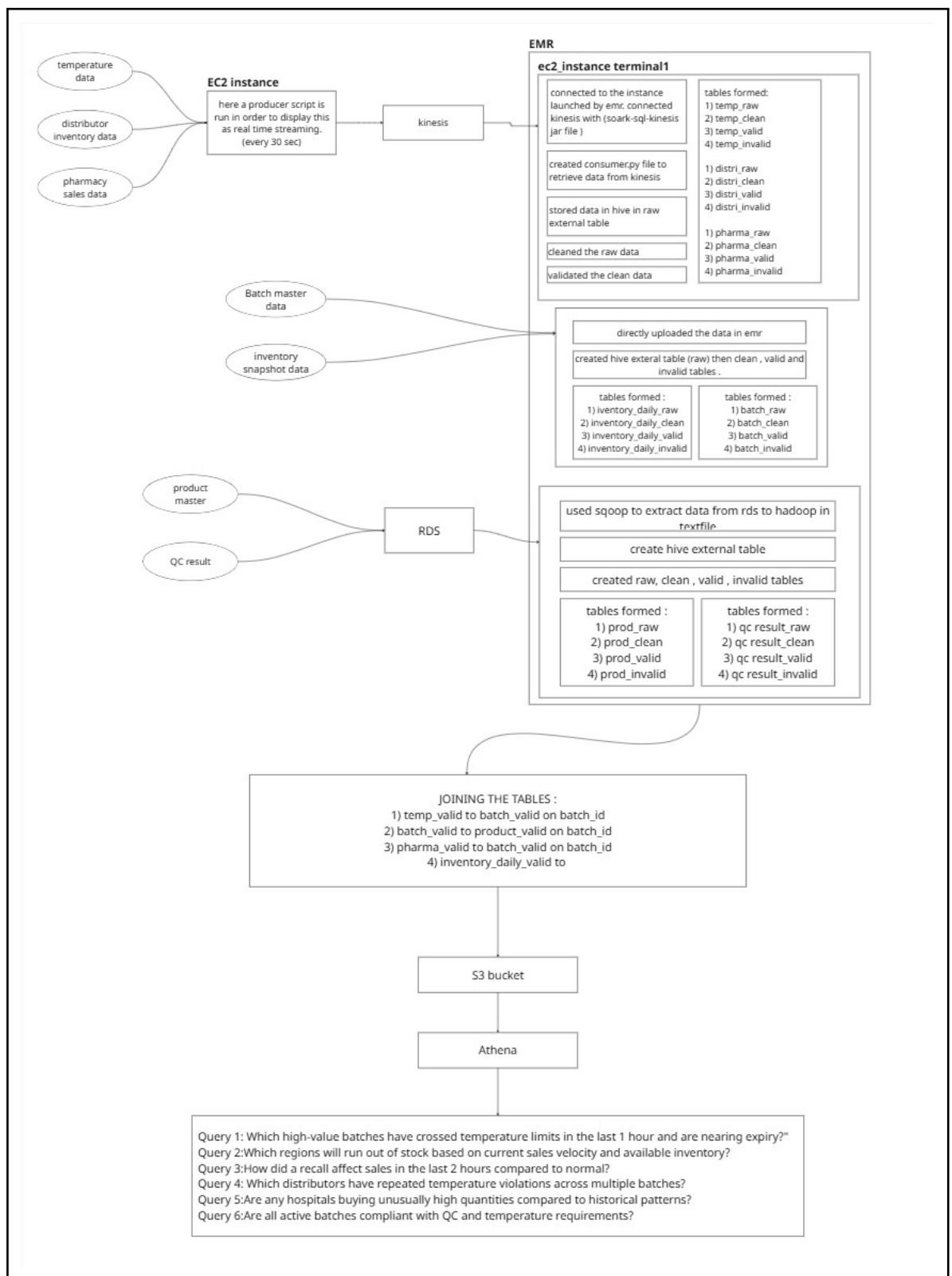


Figure 2.6: Detailed Architecture of Pharmaceutical Supply Chain Platform

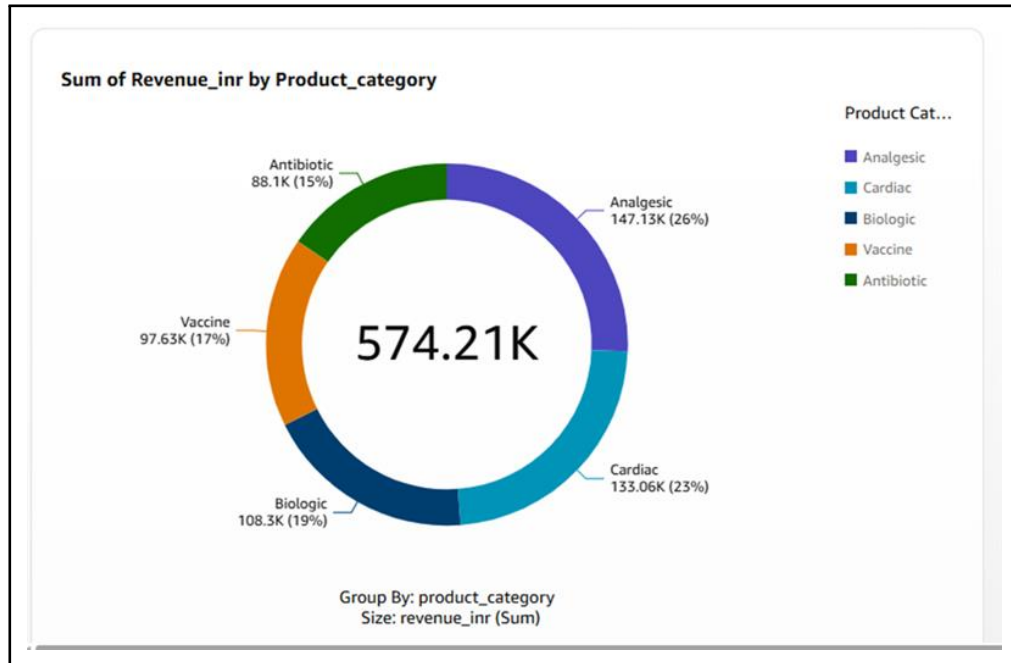


Figure 2.7: QuickSight Dashboard - Cold Chain Compliance and Regional Sales Analysis

- 183 valid records
- 17 invalid records
- Datasets exported to S3 in Parquet format.
- Successful creation and querying of Athena tables.

Dashboard Results:

- Athena queries performed efficiently across applications.
- QuickSight datasets (8,400 rows) imported successfully with 100% accuracy.
- Dashboards displayed correctly with no errors.

3.8 Conclusion

The Pharmaceutical Supply Chain Streaming Analytics Platform project is a successful one. It presented a commercial, production-scale, dual-lane data engineering system that transforms pharmaceutical operations from reactive, batch-refreshed reporting to near-real-time decision intelligence. Spark Structured Streaming integrates with Amazon Kinesis Data Streams and Amazon EMR, using Hive, Apache Sqoop, Amazon Athena, and Amazon QuickSight.

Table 2.10: Pipeline Validation Test Cases - Pharmaceutical Supply Chain Platform

Test Case	Description	Expected Results	Status
TC-01	Kinesis producer emission	Records from all sources published	Pass
TC-02	Temperature conversion	Correct Celsius values calculated	Pass
TC-03	Temperature checking	Out-of-range values filtered	Pass
TC-04	Sales validation	Invalid quantity/price removed	Pass
TC-05	Inventory validation	Valid change types retained	Pass
TC-06	Data persistence	Streams checkpointed successfully	Pass
TC-07	Sqoop extraction	Data loaded to HDFS successfully	Pass
TC-08	Hive validation pipeline	Correct valid/invalid split	Pass
TC-09	Product validation	Invalid temperature ranges rejected	Pass
TC-10	S3 Parquet export of batch table	All 12 variants validated in S3 listing	Pass
TC-11	Athena cross-domain join on <code>batch_id</code>	Streaming and batch data correctly joined	Pass
TC-12	QuickSight SPICE ingestion	8,400 rows ingested successfully; 0 rows dropped	Pass

The system addresses four major pharmaceutical supply chain issues—cold chain monitoring, batch traceability, inventory visibility, and recall impact assessment—within one unified, batch-id connected analytical platform.

This was the most technically advanced of all the internship projects. It involved real-time event streaming, Kinesis producer and consumer design, Spark Structured Streaming integration with Hive, JDBC-based batch extraction using Sqoop, multi-tier Hive ETL with complex business rules, cross-domain analytical queries in Athena, and cloud-native BI delivery via QuickSight. Together, streaming and batch processing form a unifying platform that reflects the Lambda Architecture pattern embraced by enterprise-level pharmaceutical data platforms.

Compliance and business-wise, the platform produced tangible results across four dimensions: - Breach detection latency in cold chains reduced to under three minutes, ensuring regulatory-grade continuous compliance evidence. - End-to-end batch traceability achieved by connecting seven datasets through computational batch joins. - Recall impact assessment time shortened from 24–72 hours to seconds via Athena cross-domain joins. - Real-time inventory coverage computation enabled proactive replenishment decisions.

BIBLIOGRAPHY

1. Chambers, B., & Zaharia, M. (2018). *Spark: The Definitive Guide*. O'Reilly Media.
2. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
3. Amazon Web Services Documentation. *AWS Glue and Amazon Athena User Guides*. [Online]. Available: <https://docs.aws.amazon.com/>
4. Apache Spark Documentation. *Structured Streaming Guide*. [Online]. Available: <https://spark.apache.org/>
5. Infocepts Internal Resources. *DataAscent Framework and Practice Area Guidelines*.